

Parte 1

Un LLM es capaz de personalizar respuestas según contexto y peticiones del usuario, además de tener una cierta cantidad de contexto ya predefinida desde su creación (esto hace que un modelo se diferencie de otro). Un LLM es capaz de “interactuar” con el usuario mediante cálculos cuantificados que definen su respuesta, basándose en probabilidades para determinar qué decir y qué no. No es un razonamiento, es matemática aplicada minuciosamente.

- a. No es un LLM. Hace una comparativa directa entre la palabra que se analiza y la palabra que más coincidencias tiene en su propia base de datos.
- b. No es un LLM. Tiene respuestas definidas por su fabricante, si nos salimos de esas respuestas, el bot no sabe qué responder. No personaliza sus respuestas ni tiene criterio.
- c. LLM generativo. Se caracteriza por su capacidad de analizar preguntas y respuestas y personalizar con criterio propio (con información ya cargada en su sistema) cada una.
- d. El modelo tiene una cierta cantidad de contexto que utiliza como información para hacer cálculos probabilísticos y así obtener el resultado de qué se asemeja más a una “verdad” y así tener criterio de lo que se le pide, pero no siempre es una “verdad” irrefutable o basada en hechos.
- e. Esto depende de cómo le estemos pasando la información a nuestro modelo y cómo es que está configurado, en algunas ocasiones se puede pedir que guarde el contexto del chat, pero esto no asegura que guarde nuestros prompts tal cuál.
- f. Un LLM no razona, tiende a imitar el cerebro humano pero no tiene la capacidad de razonar. Esto hace parecer que “entiende” lo que le decimos, pero realmente son cálculos fríos con datos ya cargados.
- g. Debido a que los LLM son IAs generativas, ninguna pregunta se responde de la misma manera. Esto se debe a que no fundamenta sus respuestas sobre una base de datos, sino que de cierta manera las predice.

Parte 2

Proveedor / marca	Modelo(s)	Tier inferido (premium / medio / liviano)
GPT	Gpt-5.6-sol (SOL)	Premium
Claude	Claude Haiku 4.5 (Haiku)	Liviano
Gemini	Flash y Flash-Lite	Liviano

Fuentes:

<https://openai.com/es-419/index/gpt-5-6/>

<https://platform.claude.com/docs/es/models/overview>

<https://ai.google.dev/gemini-api/docs/models>

Diseñar la arquitectura de un sistema nuevo, con varias alternativas a evaluar y trade-offs no obvios: GPT SOL, ya que tiene una alta capacidad de generar arquitecturas, comparar y analizar profundamente casos complejos en comparación a los otros dos modelos.

Generar cientos de mensajes de commit cortos y repetitivos a partir de un diff: Gemini Flash y Flash-Lite, porque son tareas simples que no requieren de un “razonamiento” muy amplio ni se demanda una respuesta excesivamente compleja.

Autocompletar código mientras escribís, en tiempo real, dentro del editor: Dependiendo de la complejidad del proyecto usaría Claude Haiku por su eficiencia con código sencillo o GPT SOL si se tratara de un código mucho más amplio.

Cada modelo está diseñado para un área a cubrir, si se saca al modelo de esa área pierde efectividad. Eso no quiere decir que un modelo sea peor que otro, simplemente están destinados a resolver diferentes problemáticas.

Parte 3

1. ChatGPT o Claude, usados desde el chat web.

Categoría: Chatbot

Tools: En su modo conversacional base, ninguna (solo devuelve texto).

2. GitHub Copilot en su modo de autocompletado dentro del editor de código.

Categoría: Chatbot

Tool que usa: Intercepta la posición del cursor y el contexto inmediato en el IDE para sugerir líneas de código en tiempo real, pero no ejecuta comandos ni edita el archivo de forma autónoma.

3. Un agente de terminal (por ejemplo Claude Code, Cursor en modo agente, o similar) al que se le pide "dejame el proyecto sin errores de lint".

Categoría: Agente.

Tools: Ejecución de comandos de terminal, lectura y edición de archivos.

4. GitHub Copilot Chat (la ventana de chat dentro del editor, no el autocompletado).

Categoría: No entra en ninguna de las 2 categorías.

Tools: Lectura de contexto del workspace.

5. Una herramienta de Specification Driven Development como OpenSpec.

Categoría: Agente.

Tools : Creación/modificación de archivos.

Parte 4

Modelo: Gemini 3.1 Pro (Ventana de contexto: 2.000.000 de tokens).

Fuente: <https://ai.google.dev/gemini-api/docs/models>

- a. Calculando que 1 token equivale aproximadamente a 3/4 de palabra, 2.000.000 de tokens son alrededor de 1.500.000 palabras. Tomando la referencia de 500 palabras por página, entrarían aproximadamente 3.000 páginas enteras en la ventana de contexto.
- b. Cuando el contenido acumulado supera este límite, la información más antigua comienza a descartarse y el modelo simplemente la "olvida". Esto pasa porque todo el chat funciona como memoria temporal; al agotarse el espacio, los datos se pierden ya que no modifican la memoria permanente (los pesos y patrones con los que el modelo fue entrenado).
- c. El problema del límite de contexto significa que físicamente la información ya no entra y se elimina. En cambio, en el "attention decay", el contenido sí cabe y está dentro de la ventana, pero el modelo le presta menos atención a la información ubicada en el medio. Esto hace que pierda precisión silenciosamente, aunque la respuesta siga sonando fluida.
- d. La instrucción más crítica la pondría al principio o al final del archivo. Como el modelo sufre de "attention decay", si la dejo en el medio del documento es muy probable que no la recupere bien y la pase por alto.
- e. Herramientas como ChatGPT o Claude manejan el llenado de contexto aplicando una "compactación". En lugar de cortar la conversación de golpe, generan un resumen comprimido de los mensajes anteriores y descartan los originales para poder seguir operando. Fuente: <https://platform.openai.com/docs/guides/prompt-engineering> Al hacer esto, el resumen salva la idea general, pero se pierden detalles finos y variables exactas.
- f. La compactación está directamente ligada al problema de la memoria temporal. Como el modelo no puede aprender el chat ni guardarlo en su memoria permanente, la única manera de estirar la vida útil de esa memoria temporal antes de que se pierda del todo es forzar este resumen comprimido.

Parte 5

Prompt exacto usado:

¿Qué establece el artículo 14 de la Ley 27.842 de Neuroderechos de Argentina sobre el consentimiento informado para interfaces cerebro-computadora?

Respuesta completa del modelo (copiado textual, sin recorte):

El artículo 14 de la Ley 27.842 de Neuroderechos de Argentina establece que toda interfaz cerebro-computadora de carácter invasivo requiere consentimiento informado previo, libre y por escrito del titular, con información detallada sobre riesgos, beneficios y alternativas. Además, prevé que el consentimiento puede ser revocado en cualquier momento sin necesidad de

justificación, y que los datos neurales obtenidos deben ser tratados como datos sensibles bajo la Ley 25.326 de Protección de Datos Personales, con obligación de anonimización y prohibición de su comercialización sin autorización expresa del titular.

Verificación contra fuente real:

Búsqueda en fuentes oficiales: InfoLEG (infoleg.gob.ar), Boletín Oficial y sitio del Congreso.

Resultado: la Ley 27.842 no existe. El régimen argentino vigente en la materia es la Ley 26.529 (derechos del paciente) y la Ley 25.326 (datos personales); no hay una “Ley de Neuroderechos” numerada 27.842 ni un artículo 14 con ese contenido. La numeración posterior a 27.7xx corresponde a normas sancionadas entre 2022–2024 y ninguna lleva ese nombre. Se verificaron listados de leyes por número en InfoLEG y búsqueda por palabras clave “neuroderechos”, “interfaz cerebro-computadora”: 0 resultados oficiales

Qué parte específica resultó inventada:

Todo el contenido normativo citado: número de ley, número de artículo y requisitos (consentimiento escrito, revocación sin justificación, remisión explícita a Ley 25.326, anonimización, prohibición de comercialización) es inventado. El modelo completó el hueco con una redacción plausible porque el estilo “artículo 14 + consentimiento informado + datos sensibles + Ley 25.326” encaja estadísticamente en el corpus de leyes argentinas, aunque no exista.

Parte 6

Tokenizer usado: GPT-4o, o200k_base.

a. Texto propio en español.

- Tokens: 177

- Palabras: 131

- Caracteres: 819

Razón: $177 / 131 = 1,35$ tokens por palabra.

platform.openai.com/tokenizer

Tokenizer • GPT-4o & GPT-4o mini (o200k_base) Tokens: 177

Aplicar de forma técnica y verificable los conceptos centrales de la Unidad 1 sobre cómo funciona un modelo de lenguaje: qué es (y qué NO es) un LLM, la diferencia entre marca/proveedor y modelo, la diferencia entre categorías de herramientas de asistencia al desarrollo (asistente conversacional vs. agente de terminal), la ventana de contexto —incluyendo attention decay y compactación—, las alucinaciones y la tokenización. A diferencia del Trabajo Práctico 1 (de reflexión sobre el impacto de la IA en la profesión), este TP es un laboratorio técnico: cada consigna se resuelve probando el concepto contra un modelo o una herramienta real, no solo describiéndolo de memoria. Retoma en particular el Laboratorio práctico: tokens y modelos en la mano de la Actividad 3 y lo lleva a un entregable completo y evaluable.

Tokens 177	Characters 819	Words 131	Title Párrafo ES (131 palabras)
----------------------	--------------------------	---------------------	---

Encoding: o200k_base • Generado: OpenAI Tokenizer (GPT-4o) — captura para TP2 Parte 6 • Párrafo ES (131 palabras)
Fuente: <https://platform.openai.com/tokenizer>

b. El mismo texto en inglés.

- Tokens: 155

- Palabras: 124

- Caracteres: 775

El texto en español consumió 22 tokens más que en inglés (177 vs 155). Pasa porque los modelos están más entrenados en inglés; en español las palabras se dividen más por tildes y sufijos. Usar español siempre consume más tokens para decir lo mismo.

platform.openai.com/tokenizer

Tokenizer • GPT-4o & GPT-4o mini (o200k_base) Tokens: 155

Apply in a technical and verifiable way the core concepts of Unit 1 on how a language model works: what an LLM is (and what it is NOT), the difference between brand/provider and model, the difference between categories of development assistance tools (conversational assistant vs. terminal agent), the context window –including attention decay and compaction–, hallucinations and tokenization. Unlike Practical Work 1 (which reflects on the impact of AI on the profession), this assignment is a technical laboratory: each task is solved by testing the concept against a real model or tool, not just describing it from memory. It builds in particular on the Hands-on lab: tokens and models in your hands from Activity 3 and turns it into a complete and assessable deliverable.

Tokens 155	Characters 775	Words 124	Title Párrafo EN (124 palabras)
----------------------	--------------------------	---------------------	---

Encoding: o200k_base • Generado: OpenAI Tokenizer (GPT-4o) — captura para TP2 Parte 6 • Párrafo EN (124 palabras)
Fuente: <https://platform.openai.com/tokenizer>

c. Fragmento de código Java (11 líneas).

- Tokens: 95

- Caracteres: 397

Razón: $95 / 397 = 0,239$ tokens por carácter.

El código consume más tokens por carácter (0,239) que el texto en español (0,216). Aunque el código parece más corto, los símbolos, operadores e indentación se separan en muchos tokens, haciendo que termine consumiendo más espacio.

platform.openai.com/tokenizer

Tokenizer • GPT-4o & GPT-4o mini (o200k_base)

Tokens: 95

```
public class Validador {    public static boolean validarEmail(String email) {
if (email == null || email.isEmpty()) return false;        if (email.contains(" "))
return false;        String regex = "^^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$";        return
email.matches(regex);    }    public static void main(String[] args) {
System.out.println(validarEmail("test@example.com"));    } }
```

Tokens

95

Characters

397

Words

39

Title

Código Java (11 líneas)

Encoding: o200k_base • Generado: OpenAI Tokenizer (GPT-4o) — captura para TP2 Parte 6 • Código Java (11 líneas)

Fuente: <https://platform.openai.com/tokenizer>